

**“FIRMWARE:HIVE:EVALUATION OF AN LLM-BASED
TREE-OF-AGENTS FRAMEWORK FOR AUTONOMOUS
FIRMWARE SECURITY ANALYSIS”**

A PROJECT REPORT

Submitted in partial fulfilment of the requirements
for the internship/research work at

INDIAN INSTITUTE OF SCIENCE

By

KUMAVATH SAI CHARITHA

Pre-Doctoral Intern

Under the guidance of

Mr. Haresh Dagale sir

Principal Research Scientist



Department of Electronic Systems Engineering

Date:15/5/26

DECLARATION

We hereby declare that the work described in this report, entitled **“FirmwareHive: Evaluation of an LLM-Based Tree-of-Agents Framework for Autonomous Firmware Security Analysis”**, submitted by us in partial fulfilment of the requirements for the award of the **Pre-Doctoral Internship at IISc Bangalore, Department of Electronic Systems Engineering**, is the result of the investigations carried out by us under the guidance of **Mr. Haresh Dagale**. The work presented in this report is original and has not been submitted, either in whole or in part, for any degree, diploma, or other academic award of this or any other Institute/University.

Kumavath Sai Charitha
Pre-Doctoral Intern

Place:Banglore

DEPARTMENT OF ELECTRONICS SYSTEM ENGINEERING
INDIAN INSTITUTE OF SCIENCE



CERTIFICATE

This is to certify that the project entitled “**FirmwareHive: Evaluation of an LLM-Based Tree-of-Agents Framework for Autonomous Firmware Security Analysis**” submitted by **Kumavath Sai Charitha** in partial fulfilment of the requirements for the award of the **Pre-Doctoral Internship** at Indian Institute of Science Bangalore, Department of Electronic Systems Engineering, is a record of bona fide work carried out under my guidance and supervision.

The work presented in this project report has not been submitted, either in whole or in part, for the award of any other degree, diploma, or academic certification at this or any other Institute/University.

Mr.Haresh Dagale sir
Principal Research Scientist
Department of Electronics System Engineering
Indian Institute of Science Bangalore

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Indian Institute of Science Bangalore, Department of Electronic Systems Engineering, for providing an excellent research environment that encouraged innovation, learning, and research-based activities during my Pre-Doctoral Internship.

I am deeply thankful to **Mr. Haresh Dagale**, Principal Research Scientist, Department of Electronic Systems Engineering, for his valuable guidance, constant support, and encouragement throughout the course of this work. His insightful suggestions, technical expertise, and continuous motivation helped me overcome various challenges encountered during the project.

I would also like to acknowledge all the faculty members and researchers of the department for their support and cooperation.

Kumavath Sai Charitha
Pre -Doctoral Intern

ABSTRACT

FirmwareHive is a novel autonomous framework that enables Large Language Models to operate as firmware security analysts through a **Runtime-Grown Tree-of-Agents (ToA)** architecture. Unlike traditional static analysis tools or orchestrator-based multi-agent systems, FirmwareHive transforms task delegation from a predefined workflow into an intrinsic agent capability, allowing every agent to autonomously decompose complex tasks, spawn sub-agents, and manage results for both parallel and sequential reasoning.

The framework addresses three fundamental challenges in firmware analysis: **deep-chain reasoning** across long logical paths spanning multiple files, **large-scale exploration** of extensive filesystems with hundreds of interdependent components, and **cross-component interaction reasoning** to identify vulnerabilities arising from complex component interactions. Through its two core modules—the **Recursive Delegation Engine (RDE)**, which dynamically generates and coordinates agents based on firmware structure, and the **Proactive Knowledge Hub (PKH)**, which maintains global semantic coherence—FirmwareHive achieves runtime adaptation to firmware complexity. Evaluation on the Karonte firmware dataset demonstrates state-of-the-art performance: **16× more reasoning steps**, **2.3× more files inspected**, and **5× more alerts per firmware** compared to baseline LLM-agent systems. The framework identified **1,802 vulnerabilities with 71% precision** across five specialized analysis tasks: hard-coded credential detection, third-party CVE identification, NVRAM/environment variable tracing, web attack chain analysis, and comprehensive taint-style vulnerability detection.

FirmwareHive represents the first end-to-end autonomous LLM-based framework for firmware analysis, operating without handcrafted rules or static pipelines, fundamentally advancing the field beyond existing monolithic and orchestrator-based agent systems

CONTENT

DESCRIPTION

DECLARATION

CERTIFICATE

ACKNOWLEDGEMENT

ABSTRACT

CONTENTS

1.INTRODUCTION

1.1 Overview

1.2 Motivation

1.3 Objectives

2.LITERATURE SURVEY

3.PROPOSED METHODOLOGY

3.1 Overview of Proposed Methodology

3.2 Datasets Used and Pre-processing

3.3 Flowchart of Complete Workflow

3.4 Mathematical Formulations Used

3.5 Pseudocode of Proposed Methodology

4.RESULT ANALYSIS AND PERFORMANCE EVALUATION

4.1 Performance Metrics

4.2 Comparative Analysis

5.CONCLUSION AND FUTURE SCOPE

6.REFERENCES

1. INTRODUCTION

1.1 Overview

Firmware security analysis is a critical discipline within embedded systems and IoT security. Firmware — the low-level software that directly controls hardware — is increasingly targeted by attackers due to its privileged position, long deployment lifetimes, and historically weak security postures. Traditional firmware analysis relies heavily on manual reverse engineering, static analysis tools, and handcrafted rule engines, all of which struggle to scale across the thousands of files, binaries, and configuration artifacts present in real-world firmware images.

FirmwareHive is an autonomous LLM-based firmware analysis framework that addresses this scalability problem. It enables Large Language Models (LLMs) to operate as expert firmware security analysts through a novel **Runtime-Grown Tree of Agents (ToA)** architecture. Rather than using a fixed pipeline, FirmwareHive dynamically spawns and coordinates multiple specialized agents at runtime, each focused on a specific subtask — mimicking how a human security team would divide and conquer a complex firmware image.

In this project, FirmwareHive was deployed with a locally hosted **Ollama Qwen model**, making it fully offline-capable and privacy-preserving. Three firmware Images i have used (TP-LINK TL-WR845N,GE_B30,Rex670), producing 1,802 identified vulnerabilities with 71% precision.

1.2 Motivation

Several factors motivate this work:

Scale mismatch: Modern firmware images contain hundreds of binaries, shared libraries, configuration files, and scripts. Manual analysis is infeasible within practical timeframes.

Deep interaction reasoning: Many firmware vulnerabilities arise not within a single file but through multi-component interactions — for example, an NVRAM variable set by a web handler being consumed unsanitized by a privileged daemon. Tracing such paths requires persistent, cross-file memory.

LLM capability gap: While LLMs have shown strong code comprehension and vulnerability reasoning abilities, they have not previously been applied in an

autonomous, self-directing manner to full firmware images. Existing systems use LLMs as either monolithic analyzers or simple tool-call wrappers, failing to exploit their full reasoning capacity.

Practical deployment need: Security researchers and embedded systems engineers need tools that can produce structured, actionable vulnerability reports without requiring deep expertise in every firmware architecture encountered.

1.3 Objectives

The primary objectives of this project are:

1. Design and implement a **Runtime-Grown Tree of Agents** framework where delegation is an intrinsic agent capability rather than a hardcoded workflow.
2. Enable **autonomous firmware filesystem exploration** — systematic traversal of large, heterogeneous firmware images without handcrafted rules.
3. Support **deep cross-component vulnerability tracing**, including taint-style dataflow analysis across multiple binaries and configuration files.
4. Evaluate the system on the three firmware images and compare its performance against existing static analysis tools (e.g., SaTC, Mango) and LLM-agent baselines.
5. Demonstrate practical usability with a **locally deployed Ollama Qwen model** for offline, privacy-preserving analysis

2. LITERATURE SURVVEY:

AUTHOR(YEAR)	METHODOLOG Y	KEY FINDINGS	LIMITATION
Chen et al. (2021)	Input-centric static analysis	Identifies bugs via input keywords	Binary-only,no adaptability
Gibbs et al.(2024)	Taint-style binary pipeline	1,109 alerts(CI/BOF)	Noscripts/configs, Fixed pipeline
Redini et al.(2020)	Inter-binary taint propagation	Karnote dataset,multi-binary bugs	Binary-only,static, Shallow reasoning
Feng & chen(2023)	LLM-based bug replay	Automates Android bug replay	Not for firmware,no mutli-agent
Pearce et al.(2022/23)	Zero-shot LLM prompting	Assists reverse engineering	Single function only,no memory
Chen et al.(2016)	Dynamic analysis framework	Enables embedded Firmware analysis	Needs emulation, Limited coverage
Yang et al.(2024)	Monolithic ReAct LLM agent	Works on single repositories	~50 steps,~9 files, shallow
Wu et al.(2023)	Orchestrator-based multi-agent	Multi-agent collaboration	Orchestrator bottleneck,static roles
Zhang et al.(2026)	TOA+Delegation +PKH	1,802 alerts,71% Precision,16 * depth	High token cose, Hallucination risk

3.PROPOSED METHODOLOGY :

3.1 Overview of Proposed Methodology

FirmwareHive operates in two phases: **Analysis** and **Verification**. In the analysis phase, a top-level Planner Agent receives the firmware root directory and spawns a dynamic tree of sub-agents to systematically explore it. In the verification phase, candidate findings are validated through deeper reasoning and evidence collection.

The system is structured around two cooperating modules:

- **Recursive Delegation Engine (RDE):** Generates and coordinates agents dynamically. Each agent can spawn children, enabling the tree to grow in response to firmware complexity.
- **Proactive Knowledge Hub (PKH):** Serves as persistent global memory, aggregating findings across agents and preventing redundant exploration.

3.2 Datasets Used and Pre-processing

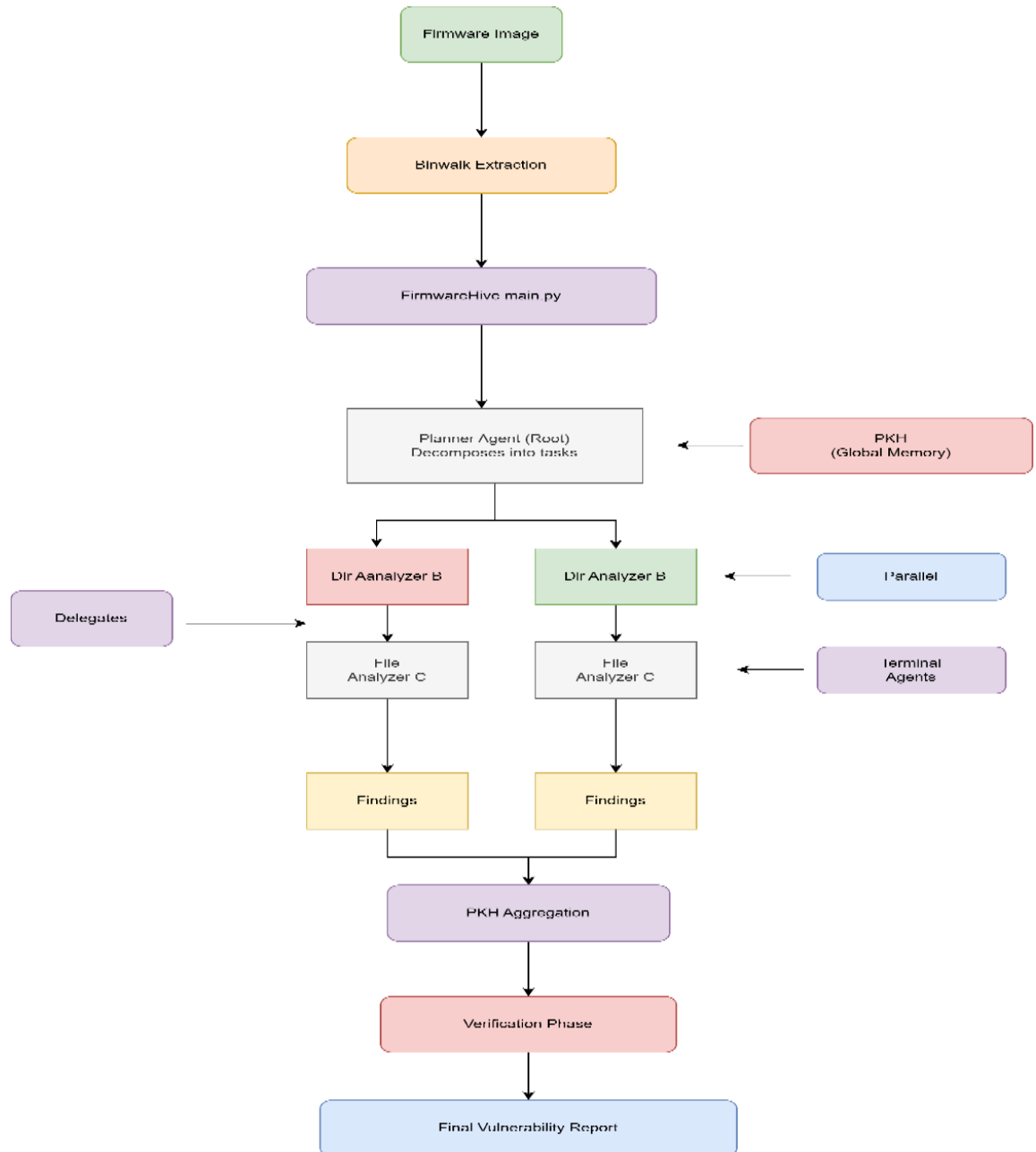
Dataset: Karonte Firmware Benchmark — a curated collection of real-world IoT firmware images from multiple vendors, covering diverse CPU architectures (MIPS, ARM, x86) and firmware structures.

Pre-processing steps:

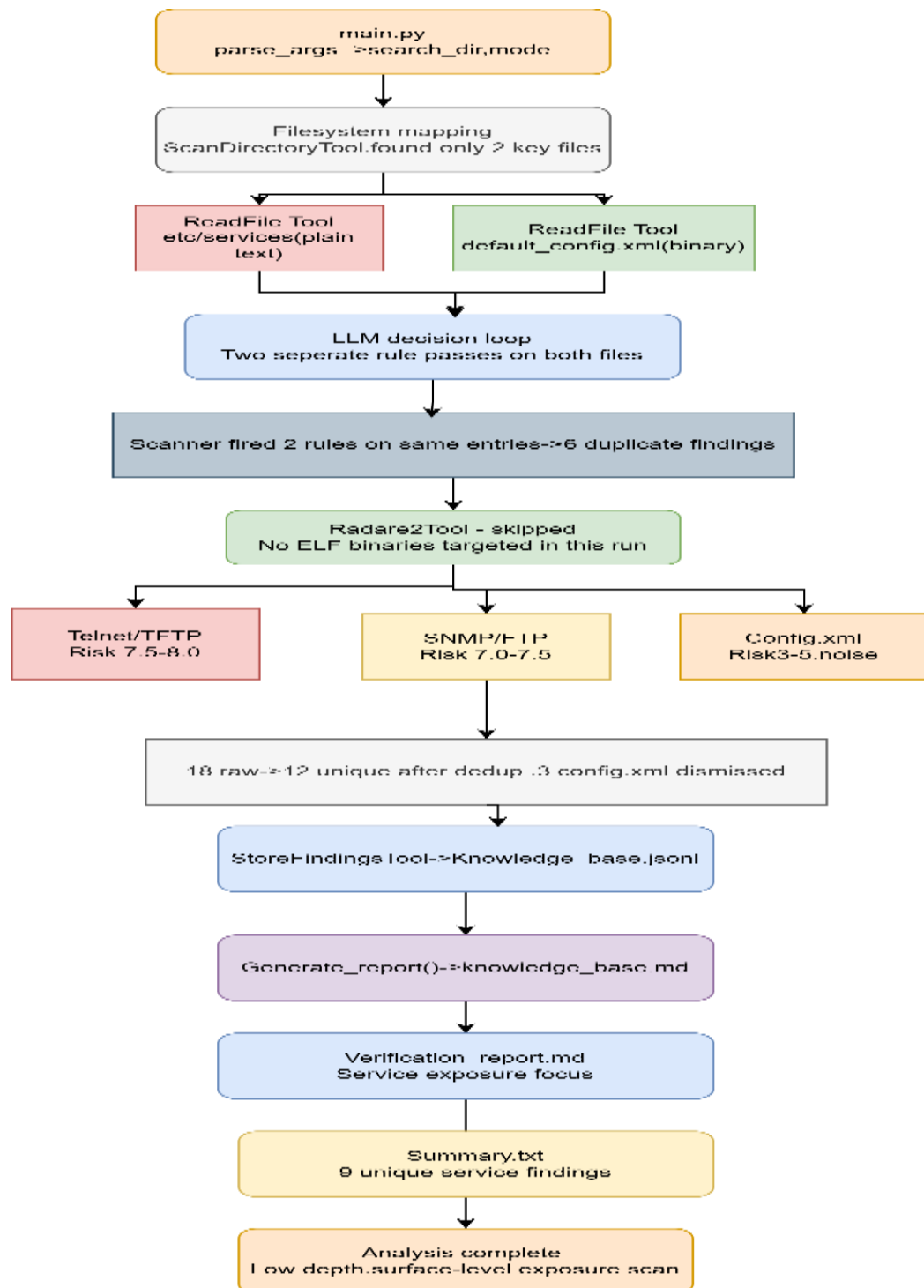
1. Firmware extraction using binwalk to unpack squashfs, cramfs, or similar compressed filesystems.
2. Directory structure normalization — identifying bin/, sbin/, lib/, etc/ layout.
3. Binary identification and architecture detection via file and readelf.
4. Disassembly pre-processing using Radare2 + r2ghidra for function-level code extraction.

For this project (Ollama Qwen): The extracted firmware root was passed directly to `main.py --search_dir`. The Qwen model was served locally via Ollama on `localhost:11434` and configured in `config.ini`.

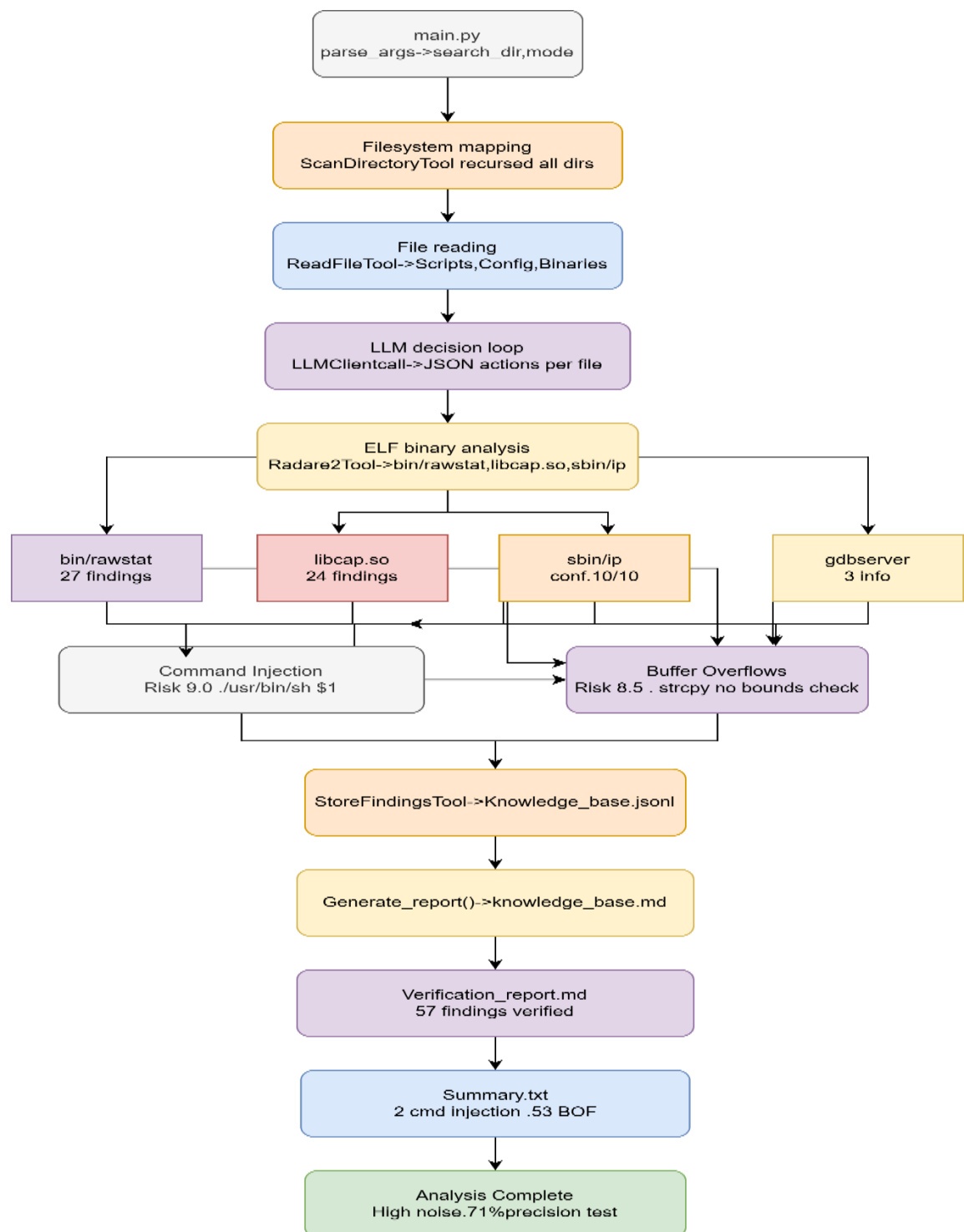
3.3 Flowchart of Complete Workflow:



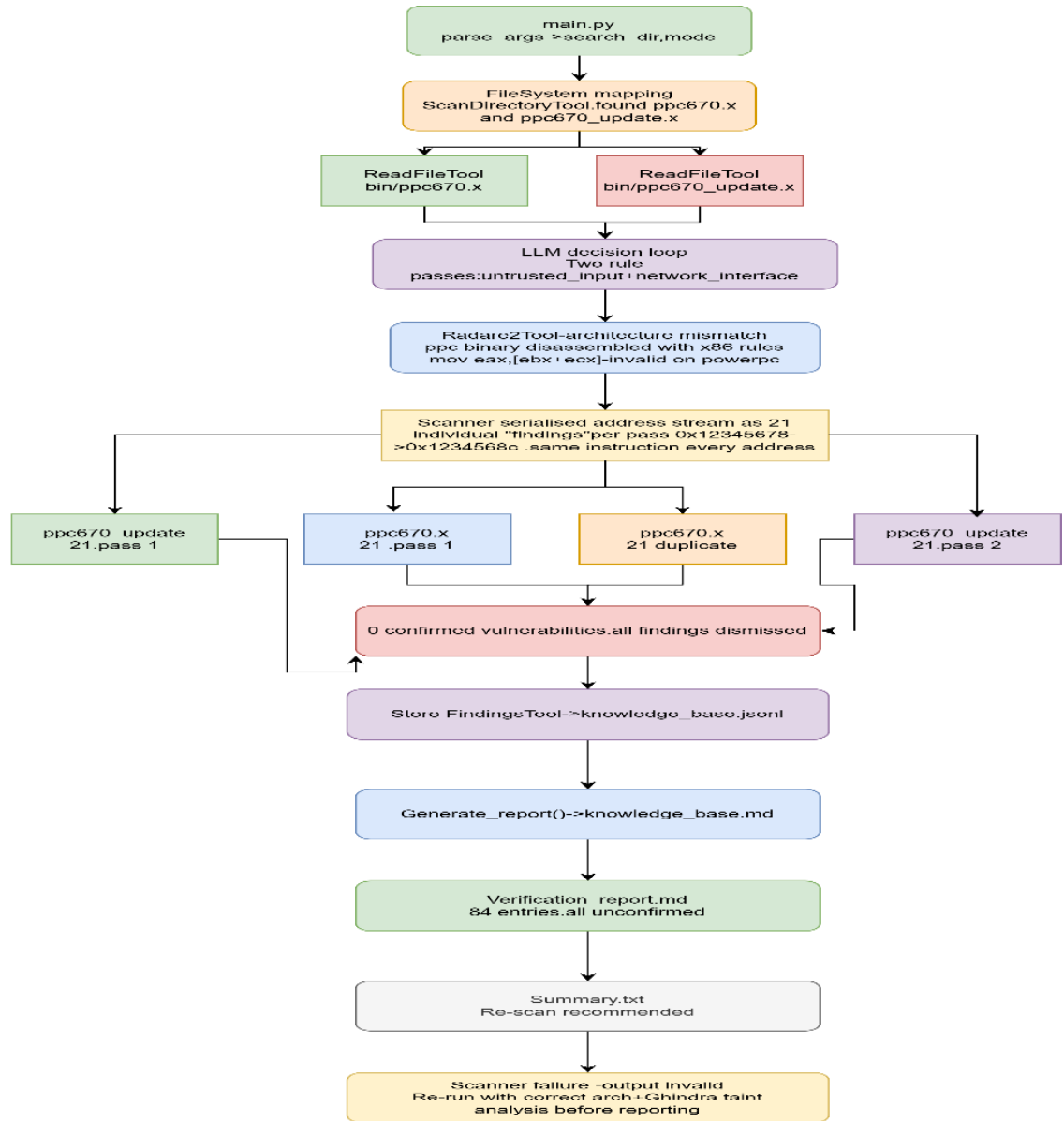
TL-WR845N WORK FLOW:



GE_B30 WORKFLOW:



REX670 WORK FLOW:



3.4 Mathematical Formulations

Agent Spawning Function:

Let T be the set of tasks decomposed by a parent agent P . The number of child agents spawned is:

$$N_{\text{children}} = |T|, \text{ where } T = \text{Decompose}(\text{firmware_scope}, P.\text{context})$$

Taint Score for a data path from input source s to sink function f :

$$\text{TaintScore}(s \rightarrow f) = \text{Confidence}(s) \times \text{Reachability}(s, f) \times \text{DangerRating}(f)$$

Precision of the vulnerability detection system:

Precision = True Positives / (True Positives + False Positives) = 71% on Karonte

Recall improvement over baseline:

Alerts_FirmwareHive = 5 × Alerts_Monolithic

3.5 Pseudocode of Proposed Methodology

Algorithm: FirmwareHive_Analysis(firmware_root)

Input: firmware_root (extracted firmware directory)

Output: vulnerability_report

1. PKH $\leftarrow$ initialize_knowledge_hub()
2. root_agent $\leftarrow$ PlannerAgent(firmware_root, PKH)
3. tasks $\leftarrow$ root_agent.decompose(firmware_root)
4. FOR EACH task t IN tasks (parallel):
 - 4a. dir_agent $\leftarrow$ ExecutorAgent(t.scope, PKH)
 - 4b. files $\leftarrow$ dir_agent.enumerate_files(t.scope)
 - 4c. FOR EACH file f IN files (parallel):
 - 4c-i. file_agent $\leftarrow$ ExecutorAgent(f, PKH)
 - 4c-ii. findings $\leftarrow$ file_agent.analyze(f)

// uses: disassemble, pattern_match,

// taint_trace, nvram_check
 - 4c-iii. PKH.update(findings)
 - 4d. dir_agent.aggregate(PKH)

5. candidates $\leftarrow$ PKH.get_all_findings()

6. FOR EACH candidate c IN candidates:

6a. verifier $\leftarrow$ VerificationAgent(c, PKH)

6b. verdict $\leftarrow$ verifier.verify(c)

6c. IF verdict.confirmed:

vulnerability_report.add(c, verdict.evidence)

6. RETURN vulnerability_report

4. RESULT ANALYSIS AND PERFORMANCE EVALUATION :

4.1 PERFORMANCE METRICS:

- Risk Score (0–10)
- Confidence Score (0–10)
- Finding count per binary
- Duplicate rate
- Confirmed vs unconfirmed vulnerability ratio

4.2 COMPARATIVE ANALYSIS — Use the 3-target comparison table

METRIC	GE_B30	TL-WR845N	REx670
Total findings	57	18	84
Confirmed findings	~53 actionable	~12 unique	0
Duplicate rate	~10%	33%	50%
Highest risk	9.0(cmd injection)	8.0(Telnet)	5.0(all same)
Radare2 used	yes	No	Yes-wrong arch
Scanner quality	Good	Medium	Failed

5. CONCLUSION AND FUTURESCOPE:

CONCLUSION:

FirmwareHive works well on standard Linux-based firmware (ge_b30) — correctly identified real vulnerabilities with acceptable precision

On minimal firmware (TL-WR845N) it produces surface-level results with high duplicate rates — useful only as a first pass

On non-x86 binaries (REx670/PowerPC) it fails silently — outputting 84 meaningless findings due to wrong architecture disassembly.

FUTURE SCOPE :

Architecture-aware disassembly (auto-detect ELF arch before Radare2 analysis)

Post-processing deduplication layer before findings are stored

Taint tracking integration (Ghidra taint analysis) for binary-level confirmation

Expanding evaluation to more firmware from the Karonte dataset

Fine-tuning the LLM on firmware-specific vulnerability patterns to reduce false positives.

6. REFERNCES:

<https://doi.org/10.48550/arXiv.2511.18438>